

Reducing CPU bottlenecks in small-model-big-data inference using Pytorch and CUDA

This copy of the report is just for drafting and record-keeping purposes and not for final submission as an internship report.

1. Introduction

1.1. AI adoption and AI infrastructure

The use of learned models in enterprise, military, and consumer applications continues at a brisk pace. This has resulted in an increased demand for XPU's (GPU's, TPU's, and other ML accelerator hardware) and a rapidly increasing global power consumption.

While options are being examined for both the increased energy demand and the increased silicon demand, it is also important to increase hardware utilization efficiency. Newer techniques, such as model graph compilation, model quantization, data and model sharding, and higher-efficiency data pipelines need to become standardized best practices across organizations.

This is also a consideration for the democratization of AI – at present, the largest and most sophisticated generative AI tools are deployed and controlled by a small number of multinational firms. The cost of hardware is a primary factor in the decision to rent (via API or via server) instead of buying. An important metric is the rate of queries that hardware can support, and how long it would take to amortize the capital expenditure.

For a developing nation like India, capital for emerging technology is still scarce since that need competes with other very real challenges, including basic infrastructure and social welfare. This makes efficient AI even more important in our context, since the resources available for AI-for-public-good projects is more limited.

1.2. A specific use case – small model, big data

This occurs frequently e.g. satellite image analysis, high-resolution histopathology etc. Here, the model size and FLOPs are modest – suitable for edge environments. Therefore, factors other than GPU compute time begin to dominate the overall time per sample (see sections 3, 4).

The end cause for GPU idling is data starvation. This occurs when the data loading (to VRAM) time per sample is greater than the GPU compute time per sample. The GPU remains idle in the remaining time. This idle time is the end symptom, not the root cause of GPU data starvation.

1.3. Contribution of this work

This work addresses a specific source of GPU starvation in Pytorch-based workloads. This is done through the analysis of Pytorch source code, and of traces from the Nsight Systems profiler and the Pytorch profiler. Experiments are conducted to validate the performance improvement on a few models with the Imagenet dataset. The specific bottleneck is the repeated copying of each batch from shared memory to pinned memory in the main process. This is eliminated by using pinned-shared memory. After this change, no data loading operations other than the final host-to-device transfer occur in the main process.

2. Test setup

2.1. System environment

All testing was performed on a Runpod virtual machine with the following configuration:

- GPU: RTX 4090 with 24GB VRAM (single GPU instance)
- CPU: 12 vCPUs
- RAM: 100 GB
- Storage: 200 GB
- Docker Image: runpod/pytorch:2.8.0-py3.11-cuda12.8.1-cudnn-devel-ubuntu22.04-test
(version as of May 2025)
- OS: Ubuntu Server 22.04
- CUDA: 12.8
- Python: 3.11
- Pytorch: 2.8 nightly build as packaged in docker image
- NVIDIA Nsight Systems: 2025.3.1

2.2. Inference task

2.2.1. Dataset

The Imagenet-1k (ILSVRC 2012) validation set was used. The version from Huggingface was used after being preprocessed and cached in storage.

The preprocessing steps are the same as Torchvision library models' preprocessing steps for these models. The preprocessed images are written to storage to remove the CPU overhead of steps like JPEG decoding and dtype casting.

Preprocessing steps:

- Decode JPEG to 3-channel tensor of uint8s.
- Reshape to (3, 256, 256)
- Center-crop to (3, 224, 224)
- Convert to float16 dtype with scaling.
- Normalize
- Write resulting tensor to storage

2.2.2. Models

Mobilenet v3 Large and Efficientnet B2 were chosen for their high accuracy and low computational footprint, and because their Torchvision implementations can be easily compiled. Ordinarily, during a forward pass, CUDA kernels are launched individually, sequentially, by the CPU, for computation on the GPU. This creates a communication overhead that results in GPU idle time. When the model is compiled to a CUDA graph, the entire sequence of model kernels is queued on the GPU from a single launch command. (See Fig 1).

2.3. Profilers

2.3.1 Nsys

Nsys primarily traces Nvidia computation and tracing APIs, along with the actual hardware execution. The main ones traced here are CUDA and NVTX (which is used by Pytorch and Nsys to annotate Pytorch-CUDA functions).

Here, the useful information provided by Nsys is the sequence of CUDA commands, the time for different computation steps, and the mapping from when a CUDA call is made by the host, to when it executes on the GPU. Also useful is information on utilization, bandwidth, etc.

Unfortunately, most of the advanced functions of Nsys require root access to the target machine, since the collection is invasive. This is not provided in default configurations by cloud services like Runpod. Further, in the limited Nsys tracing configuration (CUDA, NVTX), many CPU-side activities are not recorded.

2.3.2 Pytorch Profiler

The Pytorch Profiler is used precisely to make up for the blind spot of Nsys – it provides detailed information in its trace about the execution stack of Pytorch/Python commands. However, GPU execution information is limited. Further, the Pytorch Profiler is prone to memory leaks and crashes. It is resource-intensive and should be used only for investigating the underlying code's structure, not for timing-related information. For that, Nsys tuned for minimal overhead is sufficient.

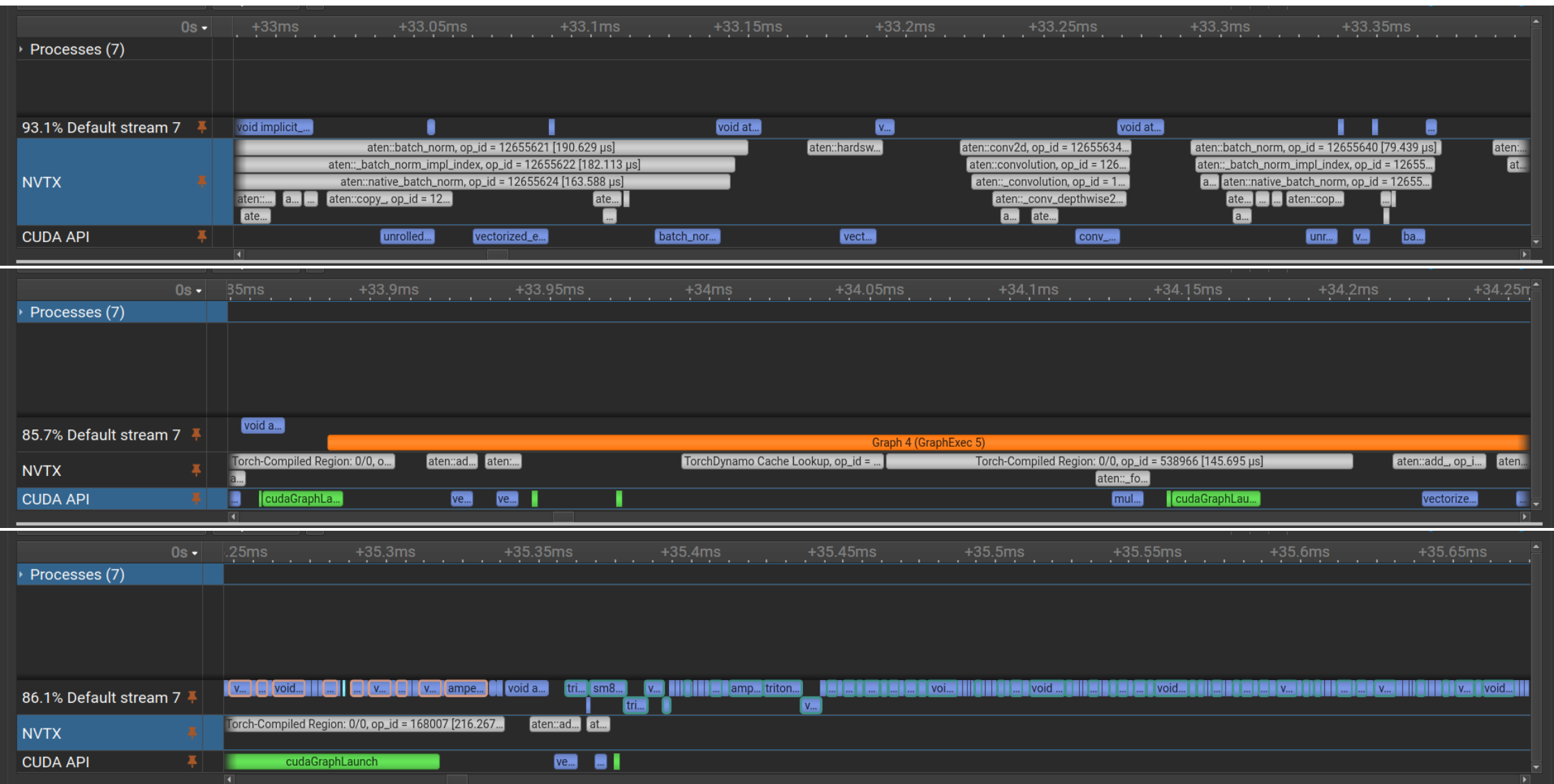


Figure 1: Three nsys traces for the same model's inference. "Default stream 7" is work on the GPU. "CUDA API" shows CUDA Runtime calls being made by the host (CPU) program. "NVTX" shows the Pytorch C++ backend calls that lead to those CUDA API calls. In the first figure, the model is not compiled, and we can see NVTX and CUDA calls for each operation e.g. convolution, batch norm etc. The second trace shows the same model running compiled as a CUDA graph, and the third trace shows the packed operations within the same CUDA graph. Notice the change in GPU utilization between the first and third graphs.

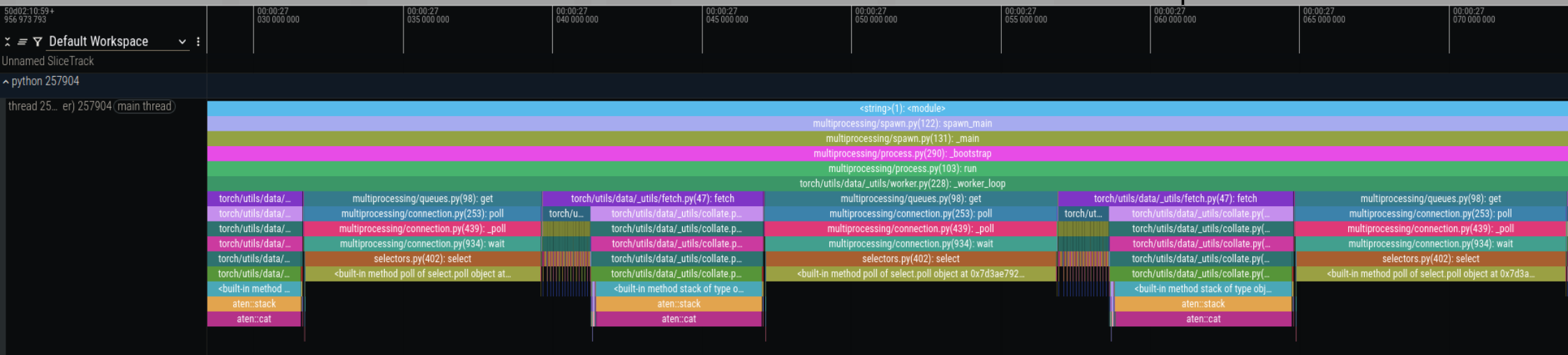


Figure 2c: Pytorch Profiler trace for the worker process for the same.

3. Analysis of the Pytorch dataloader

3.1. Traces

3.1.1. Nsys

See Fig 2a. This horizontal time-series consists of multiple rows for different devices, streams, processes, threads and even APIs. In CUDA HW, orange, purple and teal represent CUDA Graph computation, individual CUDA kernel computation, and host <-> device data transfer. In this case, the graph is the model and the data being transferred is the batch.

In thread 205444, we see the NVTX annotations and the CUDA requests as they are initiated at the CPU. Notice the absence of any markers in thread 206418 “pt_data_pin” . This will be relevant once we see the Pytorch Profiler trace.

Process 205986 “pt_data_worker” is one of the multiprocessing dataloader workers. It is responsible for generating batches. The sixteen dot-like markers here represent loading 16 samples from storage, and then they are stacked to make a new batch tensor. Here, a worker takes roughly 16.2 ms to make a batch. We have 12 workers, so the fetch time per batch should be roughly 1.35 ms. By contrast, the CUDA graph takes 0.5 ms to run and the H2D transfer takes 0.25 ms.

3.1.2. Pytorch Profiler

See Fig 2b. The Pytorch Profiler trace here focuses more on the stack on the CPU end. In the main thread, the stack descending from nn.Module is the sequence for preparing and queueing the model's CUDA graph for the next batch. `dataloader __next__` is the call to get the next batch of data – that call doesn't do anything other than pop the next batch from the queue. If the call doesn't immediately return, then it is waiting for the queue to become nonempty.

Observe the `pt_data_pin` thread. It is responsible for taking the batches created by the workers, and transferring them to pinned memory. That this happens in the main process is a critical point for later on.

See fig 2c. In this additional plot of the worker, observe the steps of making the batch. The next desired indices are first obtained from a multiprocessing queue. There are then sixteen IO read operations (batch size 16), then a tensor stack operation. Finally, the resulting batch is put in a multiprocessing queue (not visible in figure due to size).

3.2. Pinned Memory

The default memory management strategy for host RAM is to use paged memory. Here, the memory is maintained as a large number of small chunks called pages. This, along with virtual memory, gives the following enhancements in functionality:

- Different programs can have independent virtual contiguous address spaces, derived from fragmented physical memory ranges.
- Virtual memory can be larger than the physical memory available, by utilizing a part of storage for the additional memory requirement.

While this is convenient for typical use cases, it is problematic for repeated sequential data transfers, especially since the address mapping can change over the course of program execution.

It is possible, however, to request page-locked or pinned memory from the operating system. The CUDA Runtime API (CUDART) does this. The attributes of CUDA-requested pinned memory:

- Pinned memory is contiguous and static – the address mapping does not change over time.
- The CUDA runtime is aware of the physical address ranges for pinned memory requested by it.
- Host <-> device transfers from pinned host memory have a significantly higher throughput, routinely achieving the theoretical peak speed provided in official specifications.

Pytorch has a caching allocator for pinned memory to enable reuse. This is to reduce the overhead and blocking synchronization that comes with repeated CUDA calls to allocate and free host memory.

3.3. Steps of the Pytorch dataloader

This description is based on the source code as well as the traces obtained.

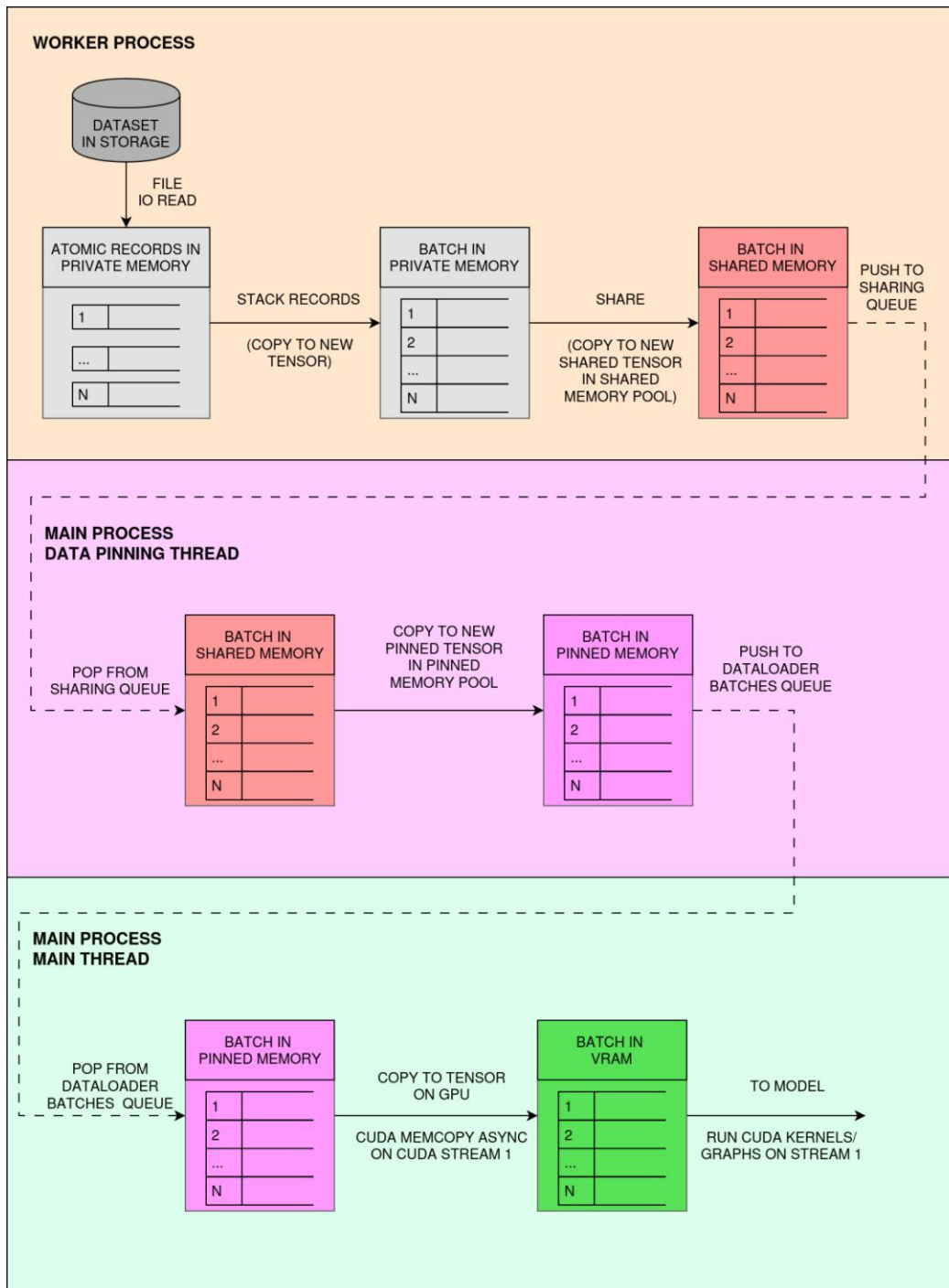


Fig 3: Depiction of data flow and processing steps for the Pytorch dataloader

3.3.1. Workers

For each batch,

- Get a list of indices from the indices queue.
- Fetch those samples from the Dataset object.
- Create new batch tensors, and then write these samples to those tensors (the 'stack' operation).

- Add these batch tensors to the worker data queue. This moves the batch to shared memory. This requires a copy operation.

3.3.2. Pin memory thread

For each batch:

- Get one shared memory batch from the worker data queue.
- Transfer this batch to pinned memory.
- Put this batch in the main data queue.

3.3.3. Main thread:

For each batch:

- Pop one batch from the main data queue.
- Copy this batch to VRAM.
- Evaluate the compiled forward pass and the accuracy metric over this batch.

3.4. General Observations:

1. The time the GPU was active for, is a mere 6.6% of total duration. This confirms GPU data starvation.
2. There are multiple host RAM $\leftrightarrow$ host RAM copy operations per batch that add to the time taken. While IO and H2D operations can happen asynchronously without involving the CPU, host RAM to host RAM copy typically involves the CPU.
3. The limiting factor appears to be the memory pinning step – while the other data loading steps can be distributed across multiple processes, pinning of the batches and H2D transfer are a single-process, single-core operation here.
4. H2D transfer and CUDA Graph execution are not happening in parallel, despite provisions existing for the same.

4. Pinned-Shared Memory Dataloader

4.1 Discussion

The key pain point is the pinning of data. One alternate approach could be to let the workers pin the data themselves. While this is possible, each worker would have to instantiate its own separate CUDA contexts and instances of the CUDA runtime. This has a high RAM overhead of at least 2GB per process, and there would also be CUDA IPC-related complications (Windows support for CUDA IPC is somewhat poor, for example). This is also why we don't let the workers handle the H2D transfers on their own.

One subtle detail is that for high-speed H2D transfers, the pinned memory needs to be pinned only with respect to the CUDA Runtime and the corresponding host process. This means that the workers can handle memory that is pinned with respect to the main process, as if it is regular paged memory.

What is used, therefore, is pinned-shared memory: memory that is shared between the main process and a worker process, and is also pinned on the main process by CUDA. This way, indirectly, copying data to pinned memory can be done by the workers, without launching additional CUDA runtimes.

4.2. Making Pinned-Shared memory

Pytorch does not provide native methods for creating pinned-shared memory. That said, Pytorch has very robust mechanisms for shared memory that are still desirable and somewhat cumbersome to replicate.

CUDA provides two methods for creating pinned memory. The first is `cudaHostMalloc` / `cudaFreeHost`, where you provide the size and get the pointer to the newly allocated memory (similar semantics as C `malloc`). The second is `cudaHostRegister` / `cudaHostUnregister`, which takes the pointer and size for an existing contiguous block of memory, and then pins that block.

The process to create and use a pinned-shared tensor is:

1. Create a typical Pytorch tensor using its APIs, with device 'cpu'.
2. Using the `.untyped_storage` API, convert the underlying storage to shared memory.
3. Obtain the data pointer and the size in memory using the same `untyped_storage` API.
4. Using the wrapper for CUDART from CUDA-Python, call `cudaHostRegister` with this pointer and size.
5. Share this tensor with other processes through the usual methods – as an argument to the target function for starting the worker or using multiprocessing queues.

Note: It is highly recommended to use the 'spawn' start method for multiprocessing.

4.3. H2D transfer and GPU computation in parallel

For this, the data transfer must take place on a CUDA stream other than the default stream, and the computation and data transfer must happen on different streams (computation can happen on the default stream).

Further, CUDA events must be used to synchronize the CUDA streams and the host(not to be confused with multiprocessing or threading events).

Methods for CUDA events:

- `Event.record( rec_stream )`: Place a marker after all CUDA jobs currently queued on `rec_stream`. We say the recorded event has completed, if all the CUDA jobs queued before the marker have finished execution on the GPU. The default state of the event is completed.
- `Event.wait( wait_stream)`: If `record` was called, make all CUDA jobs placed on `wait_stream` after this call, execute only after the event has completed.
- `Event.synchronize()`: Halt the host thread this is called in, until the event has completed.

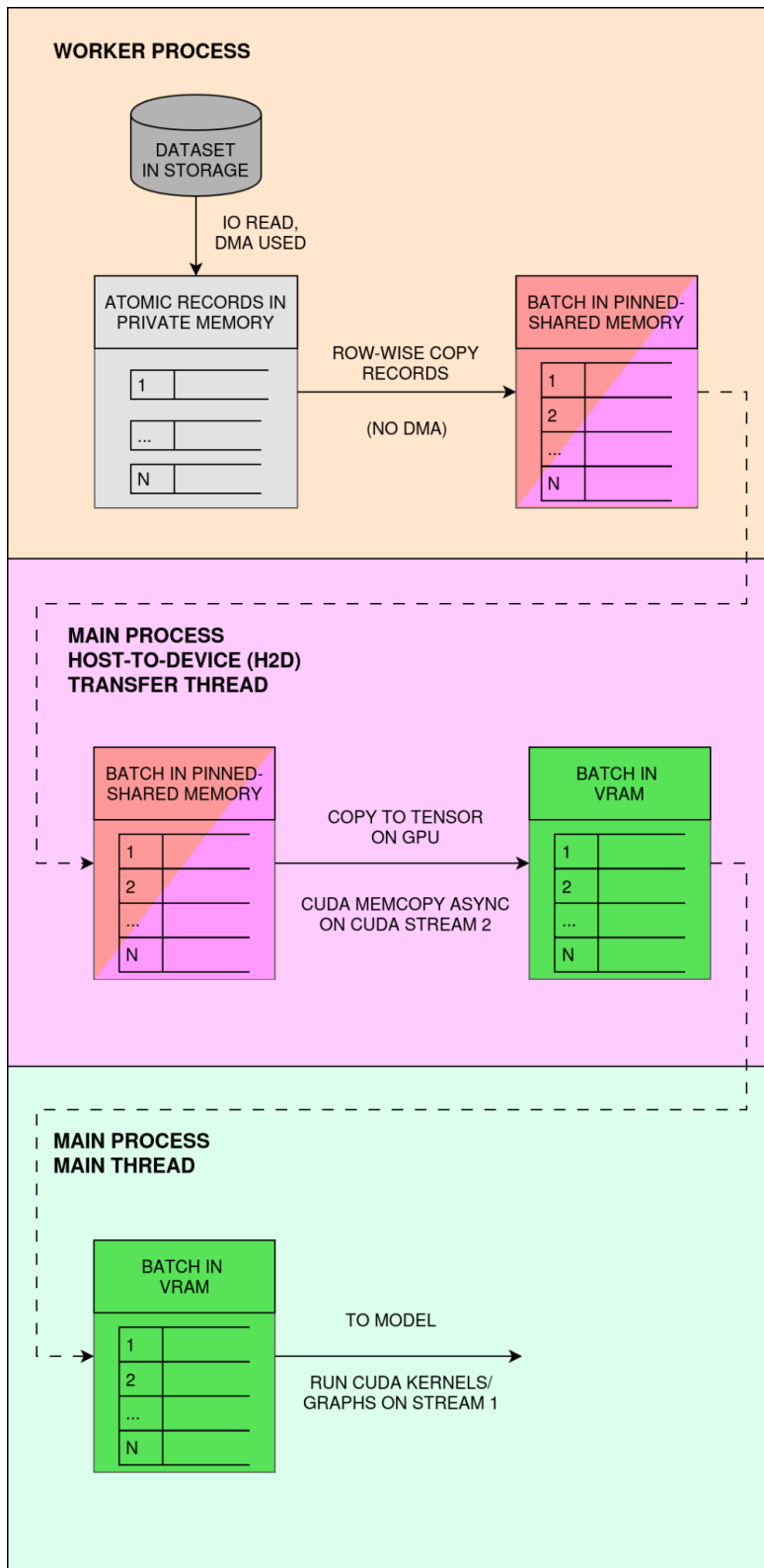


Fig 4: Depiction of data flow in the new pinned-shared dataloader

4.3. Workflow:

For each worker, there is a pinned-shared batch. There are also two batches maintained on the GPU for parallelizing compute and data transfer. There are two CUDA streams – the data transfer stream and the compute stream.

4.3.1. Worker process

The worker is provided with a pinned-shared batch of tensors, as well as multiprocessing events.

For each batch:

- Wait till it's safe to write to the batch.
- Obtain the indices for the batch.
- Fetch the corresponding sample tensors for those indices.
- Copy those sample tensors to the pinned-shared batch.
- Indicate that it is safe to read this batch.

4.3.2. Main Process, H2D Thread

Until shutdown, loop:

- Take the next pinned-shared batch (in rotation) and wait till it is safe to read from it.
- Take the next GPU batch (in rotation) and wait till it is safe to write to it.
- Perform the H2D transfer on the data
- Record a CUDA event on the data transfer stream.
- Synchronize this thread with the data transfer stream.
- Indicate that it is safe to write to this pinned-shared batch.

4.3.3. Main process, main thread:

In each iteration:

- Take the next GPU batch (in rotation) and make the computation stream wait until the transfer to it is completed.
- Evaluate the compiled forward pass and the accuracy metric over this batch, using the compute stream.
- Record a CUDA event on the compute stream, to indicate that it is safe to write to this GPU batch after computation on it is complete.

In this manner, the number of copy operations is decreased by 2.

Before: samples -> worker memory batch -> shared memory batch -> pinned memory batch -> gpu memory batch – 4 copies, (3 H2H, 1 H2D)

After: samples -> pinned-shared memory batch -> gpu memory batch – 2 copies (1 H2H, 1 H2D).

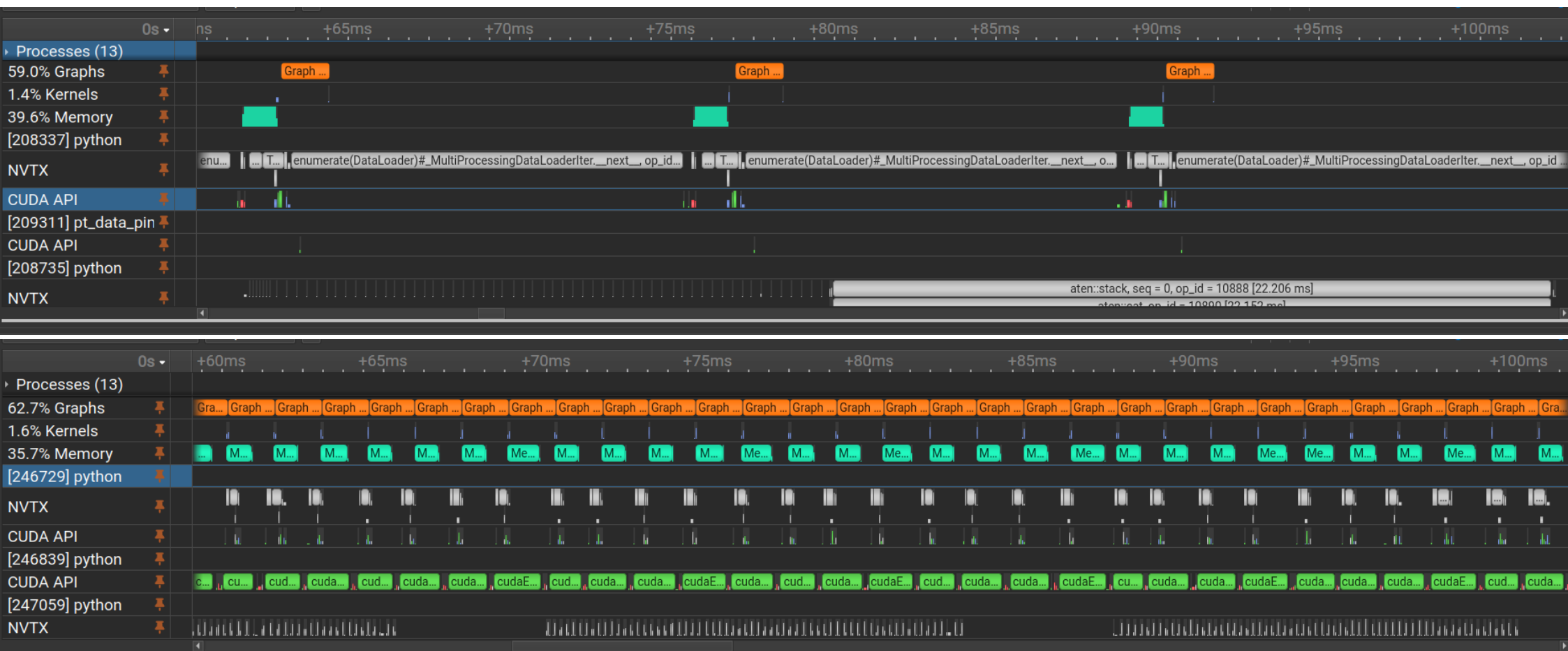


Fig 5: With the model Mobilenet v3 Large and batch size 64, the nsys traces for the Pytorch default dataloader and the new pinned-shared dataloader. Notice the difference in the density of graph calls – each block represents one batch’s forward pass.

4.4. Test protocol

The throughput was recorded at power-of-two batch sizes from 8 to 1024, for both dataloaders, for both models, without any profilers running. Low-overhead Nsys traces were also taken in the same set of configurations. Lastly, a small number of Pytorch Profiler traces were collected purely to observe the call stack, and not for any timing-related data collection.

5. Results and discussion

5.1. Results

DEF refers to the default data loader and PSH to the pinned-shared dataloader. The best throughput for each model-dataloader combination is in bold.

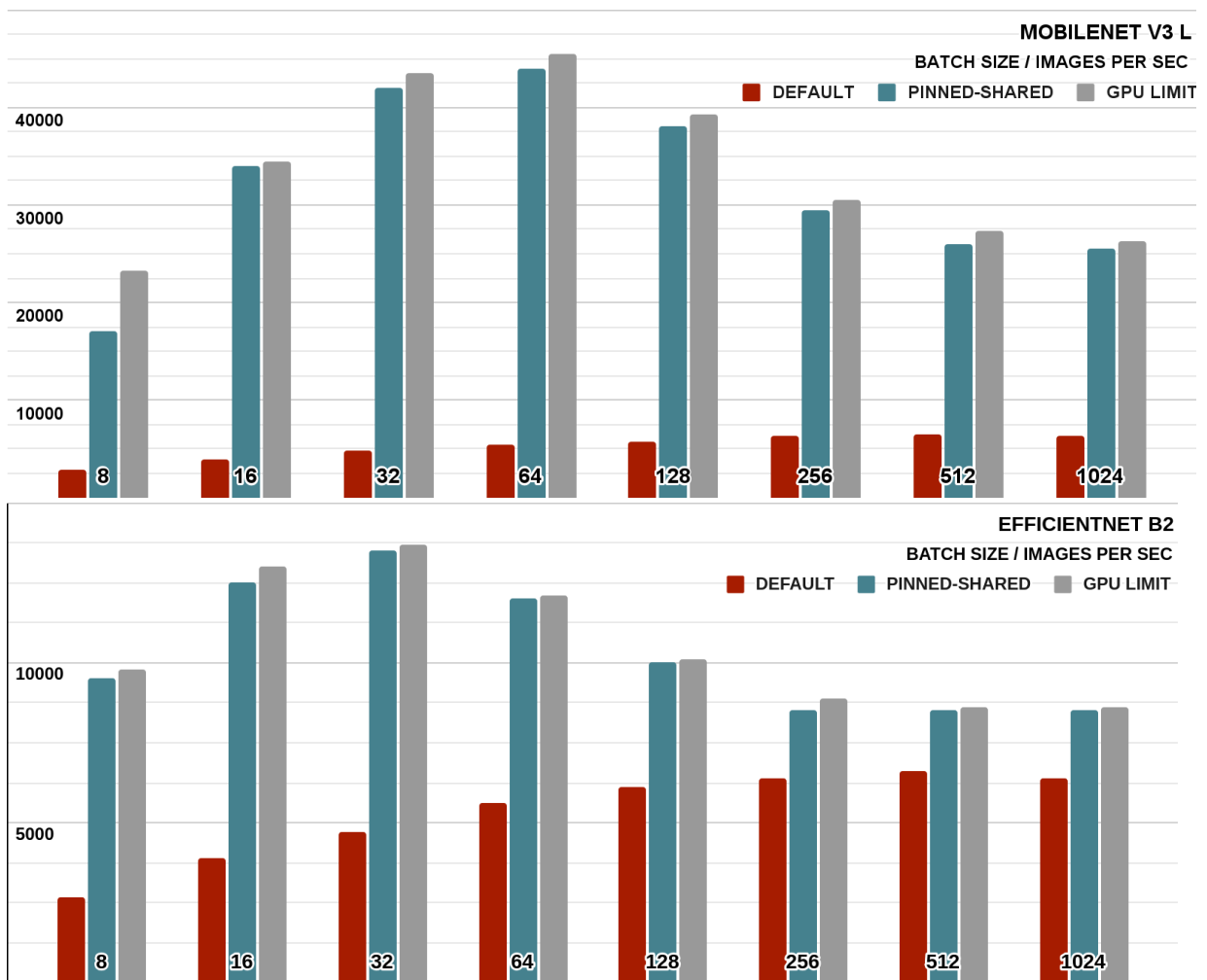
Batch Size	Throughput (images/sec)			
	MobilenetV3L		EfficientnetB2	
	DEF	PSH	DEF	PSH
8	2800	17000	2675	9500
16	4000	34000	3900	12500
32	4800	42000	4700	13500
64	5500	44000	5600	12000
128	5800	38000	6100	10000
256	6400	29500	6400	8500
512	6500	26000	6600	8500
1024	6400	25500	6400	8500

5.2. Interpreting the results

Using the pinned-shared dataloader, for Mobilenet v3 Large, the throughput increases to **6.75x** and for Efficientnet B2, to **2x** of the original throughput with the default dataloader. These results clearly demonstrate the increased computational efficiency of the pinned-shared dataloader.

What is curious is that the batch size of best throughput is 8 to 16 times smaller for the pinned-shared dataloader. The following table provides some insight.

Batch Size	GPU Compute Time and Rate					
	Mobilenet v3L			Efficientnet B2		
	Batch time (us)	Sample time (us)	Samples/sec	Batch time (us)	Sample time (us)	Samples/sec
8	345	43.13	23188	815	101.88	9816
16	465	29.06	34409	1230	76.88	13008
32	735	22.97	43537	2325	72.66	13763
64	1410	22.03	45390	5295	82.73	12087
128	3260	25.47	39264	12675	99.02	10099
256	8400	32.81	30476	28705	112.13	8918
512	18730	36.58	27336	59445	116.1	8613
1024	38890	37.98	26331	119290	116.49	8584



The GPU compute time per sample is not constant as the batch size increases. It first decreases, then continues increasing. It is interesting that the loading time for the Pytorch dataloader is the major contributor to the time per sample, instead of the GPU compute time.

5.3. Future direction

Further investigation of this is necessary to answer the following questions:

1. Can this technique be applied to larger, quantized (INT4 / INT8) models?
2. Can this technique be applied to training, and not just for inference?
3. Do other tasks and architectures (text/image/video/multimodal generation, time-series analysis etc) benefit from this technique?
4. Can facets of this technique be applied to weight offloading / KV cache offloading for large generative transformer models?